Project 2: 2D Search

Joel Batac Joshua Gilo

Technical Panel Presentation

March 8, 2026

Presentation Roadmap

1. Context and Objectives
2. Methods and Algorithm Mechanics
3. Random Growth Results
4. QR Search Results
5. Contrived Worst-Case Results
6. Cross-Section Synthesis
7. Max-Input Head-to-Head
8. Validity and Risk Assessment
9. Conclusions and Next Steps

Why 2D Binary Pattern Matching Matters

- Many practical tasks reduce to locating small binary templates inside large binary grids.
- Example domains: QR-like finder structures, symbol detection, and matrix motif search.
- This project compares seven algorithmic strategies under common workloads.
- Goal: identify which method is best under which operating condition.

Key Message

This is a scenario-driven algorithm selection study, not a single-winner contest.

Research Questions and Hypotheses

RQ1. Which algorithm minimizes equality comparisons per scenario?
RQ2. Which algorithm minimizes wall-clock time per scenario?
RQ3. How do results change from average workloads to adversarial layouts?

Hypothesis (pre-study)

- Hash/skip-heavy methods should reduce comparisons.
- Runtime leadership may diverge from comparison leadership due to implementation overhead.

Canonical Workflow and Data Sources

- Canonical notebook: `testing_notebook.ipynb`
- Sections benchmarked: Random Growth, QR Search, Contrived Worst-Case, Max-input Head-to-Head.
- Numeric truth source: raw CSV outputs recomputed by `scripts/recompute_metrics.py`.
- Slide callouts are generated from `slides/generated/metrics_callouts.tex`.

Canonical Inputs

`results_random.csv`, `results_qr.csv`, `results_contrived.csv`, `results_head_to_head.csv`

Study Scope at a Glance

Algorithms	Naive, Rabin-Karp, Vectorized RK, Greedy KMP, Greedy BM, Bird-KMP, Bird-BM
Random section size	980 rows total
QR section size	420 rows over 60 QR images
Contrived section size	49 rows
Head-to-head size	5 rows (5 algorithms)

Caveat: The local repository snapshot does not include the raw `qr/` image folder; QR evidence is chart/CSV-based.

Algorithm Family Overview

Naive	Full 2D sliding-window exact matching baseline.
Rabin-Karp	2D rolling hash over windows, then exact verification [1].
Vectorized RK	Same RK logic with NumPy-wide window hashing.
Greedy KMP	KMP filter on first row, then greedy full-row verification [2].
Greedy BM	Bad-character BM filter on first row, then greedy verification [3].
Bird-KMP	Column-encoding + Aho-Corasick-like column scan + KMP over IDs [4, 5].
Bird-BM	Same Bird preprocessing, BM over column-ID sequence.

Instrumentation: What Is Measured

- Every algorithm returns `(comparison_count, match_position)`.
- `Counter` wraps equality checks and increments on each call.
- Wall-clock is captured externally per run in the notebook harness.

Interpretation rule

comparison_count approximates filtering behavior, while **wall-clock** captures full implementation cost.

Naive Search: Core Mechanics

Input: text $T \in \{0,1\}^{N \times N}$, pattern $P \in \{0,1\}^{M \times M}$

1. Enumerate every valid top-left window (i, j).
2. Compare all M^2 pattern cells against $T[i : i + M, j : j + M]$.
3. Stop at first full match, else return `None`.

Complexity intuition

Worst-case $O((N - M + 1)^2 M^2)$. Minimal overhead, high direct comparison volume.

Naive Search: Practical Behavior

- Strength: very transparent baseline and stable implementation.
- Weakness: no pruning; cost grows quickly when no early match occurs.
- In this project: often competitive wall-clock at moderate sizes due to low constant factors.

Key Message

Naive is a crucial reference point: simple, predictable, and hard to beat when overhead dominates.

2D Rabin-Karp: Rolling Hash Pipeline

1. Compute pattern hash using row and column polynomial bases.
2. Build row-wise rolling hashes for text.
3. Build vertical rolling hashes over those row hashes.
4. On hash hit, perform exact cell verification.

Why it helps

Most windows are rejected cheaply by hash mismatch before full comparison.

2D Rabin-Karp: Caveats in This Implementation

- Hash arithmetic and rolling updates are substantial non-counted work.
- Collision verification still requires exact checks on candidate hits.
- Loop RK and Vectorized RK can show similar comparisons but very different runtime.

Caveat: Do not treat RK comparison counts as complete cost; include wall-clock for selection.

Vectorized RK: What Vectorization Changes

- Uses NumPy sliding windows and reduction operations to hash many windows together.
- Reduces Python-loop overhead in hash computation stage.
- Verification loop is still Python-level for candidate hits.

Expected effect

Lower runtime than loop RK when vectorized operations amortize setup overhead.

Greedy KMP: Two-Stage Filter + Verify

1. Build LPS array from first pattern row.
2. Run KMP across each candidate top text row.
3. For first-row matches, greedily verify remaining rows directly.

- Effective when first-row mismatches are frequent.
- Can overwork when first row matches often but full pattern fails.

Greedy BM: Bad-Character Driven Skipping

1. Build bad-character table from first pattern row.
2. Match right-to-left on that row; shift aggressively on mismatch.
3. If first-row match, verify remaining rows greedily.

Observed effect in this codebase

Greedy BM frequently delivers strongest wall-clock performance on QR/contrived tests.

Bird Methods: 2D-to-1D Reduction Strategy

1. Encode pattern columns into IDs.
2. Build automaton over unique column signatures.
3. For each top-row offset, derive a column-ID sequence of text windows.
4. Run 1D matcher over IDs: KMP variant or BM variant.

Design goal

Reduce expensive full-grid matching to structured column-sequence matching.

Bird-KMP and Bird-BM: Practical Tradeoffs

- Usually excellent in comparison-count efficiency (especially Bird-BM).
- Preprocessing and automaton traversal add heavy constant-time overhead.
- In our profiling, `_build_C_row` and `_ac_next` dominate runtime.

Key Message

Bird variants can win on comparisons yet lose on wall-clock in pure Python implementations.

Why Comparison and Runtime Rankings Diverge

Counted work	Equality checks routed through `Counter`.
Non-counted work	Hash updates, dict lookups, automaton transitions, NumPy reductions, memory traversal.
Outcome	Algorithms with fewer counted comparisons can still be slower overall.

Caveat: This is expected and not a bug: the metrics answer different questions.

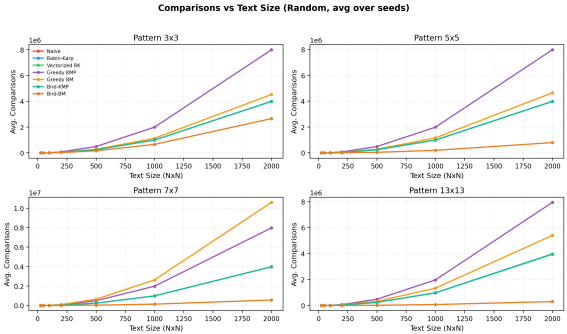
Experimental Sections and What They Stress

Random Growth	Scaling behavior under structured no-match conditions in notebook generation.
QR Search	Realistic pattern detection over downsampled QR-derived matrices.
Contrived	Near-worst-case scanning with guaranteed late match.
Head-to-Head	Maximum-size stress test revealing constant-factor realities.

Key Message

Each section reveals a different failure mode and selection criterion.

Random Suite: Bird-BM Minimizes Comparison Volume

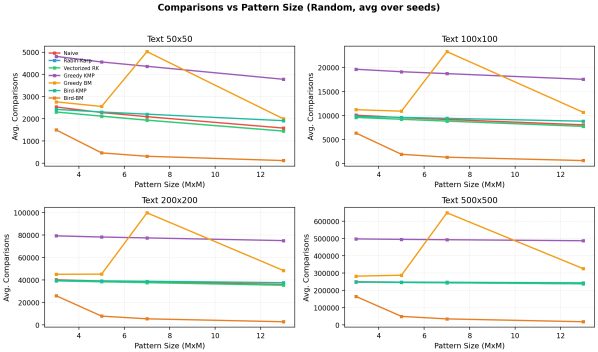


Key Message

Best average comparison count: **Bird-BM** (204743.7).

Source: `results_random.csv` and `random_comparisons_vs_textsize.png`

Pattern-Size Sensitivity Differs by Algorithm Family

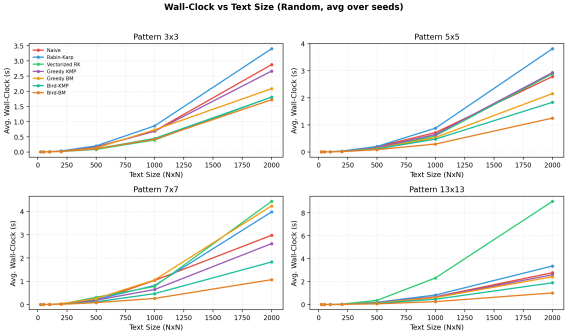


Interpretation

Bird-BM remains comparison-efficient as pattern size increases, while first-row-greedy variants scale less favorably in this suite.

Source: `results_random.csv` and `random_comparisons_vs_patsize.png`

Runtime Leader Is Not the Comparison Leader



Key Message

Best average wall-clock: **Bird-BM** (0.237957 s).

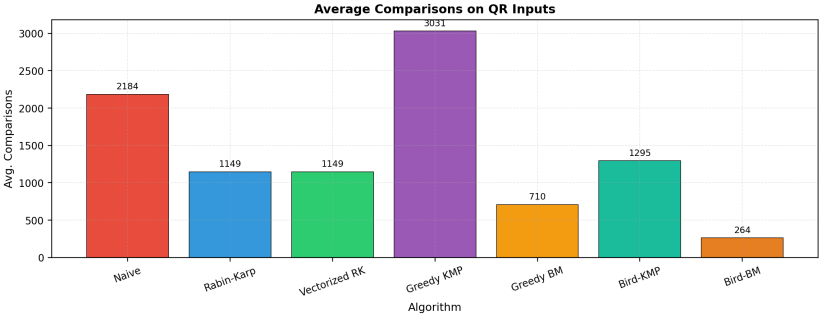
Caveat: Bird methods reduce comparisons but incur overhead that hurts runtime in this implementation. Source: `results_random.csv` and `random_wallclock.png`

Random Section Takeaway Table

Algorithm	Mean Comparisons	Mean Time (s)
Bird-BM	204743.707	0.237957
Rabin-Karp	750996.429	0.675892
Vectorized RK	750996.429	0.828940
Naive	753130.957	0.548849
Bird-KMP	754329.014	0.348045
Greedy BM	1189064.686	0.521715
Greedy KMP	1508161.414	0.508494

Caveat: Comparison and runtime rankings diverge; this motivates multi-metric selection.

On QR Data, Bird-BM Uses the Fewest Comparisons

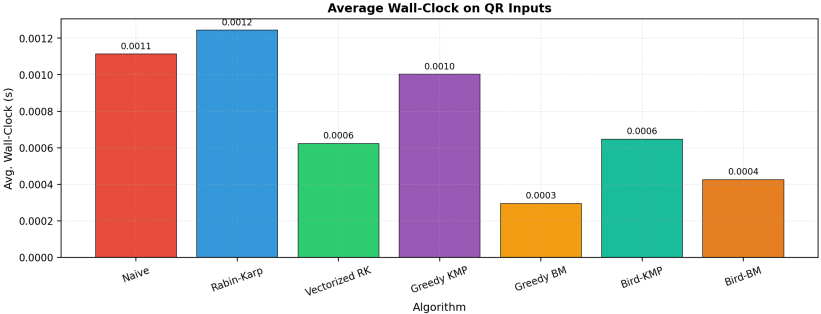


Key Message

Best average comparison count: **Bird-BM** (263.6).

Source: `results_qr.csv` and `qr_avg_comparisons.png`

Greedy BM Dominates Runtime on QR Inputs



Key Message

Fastest average runtime: **Greedy BM** (0.000295 s).

Caveat: Bird-BM wins comparisons but is slower due to implementation overhead. Source: `results_qr.csv` and `qr_wallclock.png`

Per-Image Stability Across 60 QR Inputs

Observation

The ordering is consistent across images: Bird-BM remains lowest in comparisons, while Greedy BM stays competitive with lower runtime.

Source: `results_qr.csv` and `qr_per_image_comparisons.png`

QR Section: Corrected Numeric Callouts

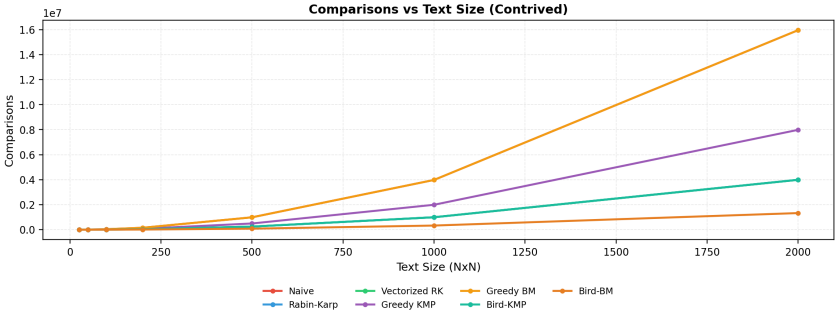
- Mean comparisons (Rabin-Karp): **1148.933**.
- Mean comparisons (Vectorized RK): **1148.933**.
- Mean runtime (Greedy BM): **0.000295** s.
- Mean runtime (Vectorized RK): **0.000623** s.

Data integrity note

These values are recomputed directly from `results_qr.csv`, superseding stale summary values.

Caveat: Raw `qr/` images are not in this repo snapshot; evidence is chart/CSV based.

Contrived Case: Bird-BM and Greedy BM Are Nearly Tied in Comparisons

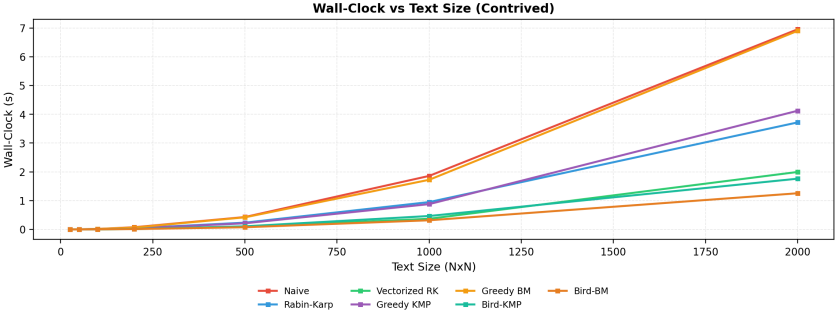


Key Message

Best mean comparisons: Bird-BM (251666.7), narrowly ahead of Greedy BM.

Source: `results_contrived.csv` and `contrived_comparisons_vs_textsize.png`

Runtime Under Adversarial Placement



Key Message

Fastest mean runtime: **Bird-BM** (0.238081 s).

Caveat: Bird-BM has low comparisons but does not translate that into runtime leadership. Source: `results_contrived.csv` and `contrived_wallclock.png`

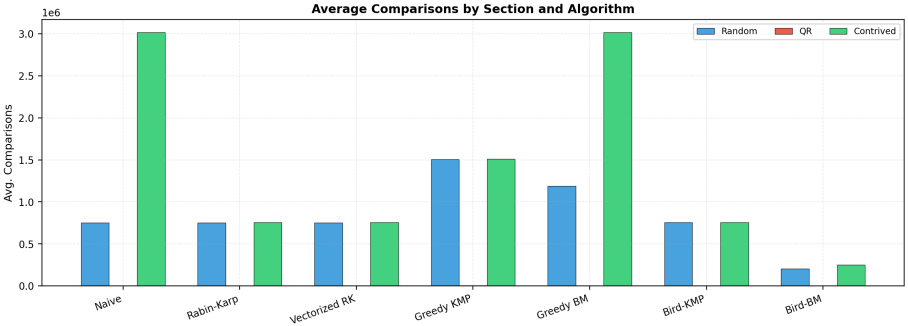
Contrived Section Summary

Algorithm	Mean Comparisons	Mean Time (s)
Bird-BM	251666.714	0.238081
Greedy BM	3019314.714	1.304504
Vectorized RK	754836.429	0.353938
Naive	3019314.714	1.334626

Interpretation

For deterministic end-of-matrix hits, skip-heavy first-row methods provide the strongest practical speedups.

Cross-Section Comparison Snapshot



Unified picture

Bird-BM repeatedly minimizes comparisons, while Greedy BM repeatedly minimizes runtime in practical scenarios.

Source: `results_summary.csv` visual plus recomputed metrics for numeric callouts

Metric Interpretation Matrix

Metric	Strength	Limitation
Comparisons	Shows filtering efficiency and candidate-pruning behavior	Misses overhead from hashing/automata/memory traversal
Wall-clock	Captures true end-to-end implementation cost	Hardware/environment dependent; less algorithmically interpretable

Key Message

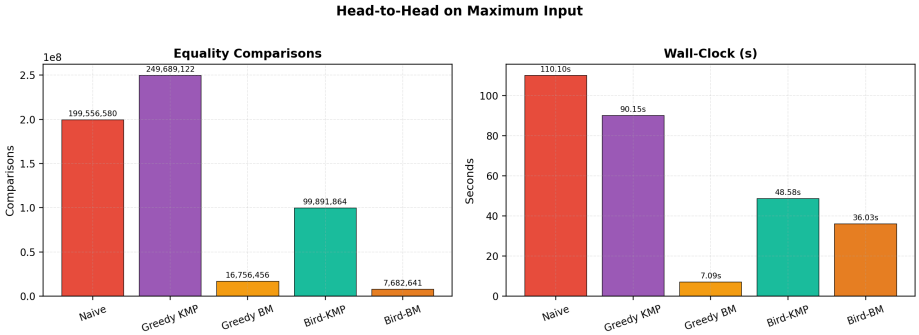
Use comparisons for *algorithmic behavior*, wall-clock for *deployment choice*.

Scenario-Based Algorithm Selection Guide

Scenario	Preferred for Time	Preferred for Comparison Budget
Random-growth style runs	Bird-BM	Bird-BM
QR-like real workload	Greedy BM	Bird-BM
Contrived adversarial placement	Bird-BM	Bird-BM

Caveat: No single algorithm optimizes both metrics in all sections.

10000x10000 Stress Test: Runtime and Comparison Outcomes



Key Message

Fastest runtime: Greedy BM at 7.0900 s; best speedup vs Naive: 15.53x.

Source: `results_head_to_head.csv` and `head_to_head.png`

Why Bird-BM Can Win Comparisons but Lose Runtime

- In this implementation, Bird-family preprocessing and traversal overhead is significant.
- The stress test magnifies constant-factor costs over huge matrices.
- Result: Bird-BM has lowest comparisons (7682641) but much higher runtime (36.0266 s).
- Greedy BM is a better practical runtime choice here (7.0900 s).

Engineering implication

Implementation detail dominates at scale; comparison-optimal is not automatically throughput-optimal.

Threats to Validity

- **Counting fairness:** comparison counters do not capture all computational work equally.
- **Data generation semantics:** “Random" section uses donut-structured text generation.
- **Artifact scope:** raw QR PNGs absent in this repository snapshot.
- **Platform dependence:** wall-clock outcomes can shift with environment/hardware.

Key Message

Conclusions are strong for this implementation stack and tested scenarios, not universal complexity laws.

Mitigations and Quality Controls

- Numeric callouts recomputed from raw CSV via reproducible script.
- Slide macros auto-generated to avoid manual transcription errors.
- Scenario-diverse design (random, QR, contrived, and max-input head-to-head).
- Explicit caveats included on empirical slides.

Recommended next validation step

Run the same harness on additional hardware and larger QR corpora once `qr/` assets are available.

Final Conclusions

1. **Comparison leader:** Bird-BM is consistently best or near-best in comparison count.
2. **Runtime leader:** Greedy BM is the most reliable wall-clock winner on tested practical scenarios.
3. **Metric divergence is real:** implementation overhead changes winner selection.
4. **Decision rule:** pick algorithm by workload and objective, not by one global metric.

Key Message

For this codebase, **Greedy BM** is the default deployment pick when runtime is primary.

Actionable Next Steps

- Standardize a unified operation-count model for better cross-algorithm fairness.
- Add profiling instrumentation to isolate overhead hotspots in Bird and RK variants.
- Expand QR corpus and retain raw assets in-repo (or with reproducible download script).
- Add CI task: regenerate metrics macros and verify slide callouts from CSV on every update.

Deliverable status

This deck is compile-ready and tied to reproducible scripts for metrics + figure preparation.

References I

R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.

D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

R. S. Bird, "Two-dimensional pattern matching," *Information Processing Letters*, vol. 6, no. 5, pp. 168–170, 1977.

A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

Backup: Random Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	204743.707	0.237957
Bird-KMP	754329.014	0.348045
Greedy BM	1189064.686	0.521715
Greedy KMP	1508161.414	0.508494
Naive	753130.957	0.548849
Rabin-Karp	750996.429	0.675892
Vectorized RK	750996.429	0.828940

Backup: QR Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	263.633	0.000426
Bird-KMP	1294.600	0.000647
Greedy BM	710.017	0.000295
Greedy KMP	3030.550	0.001002
Naive	2184.150	0.001113
Rabin-Karp	1148.933	0.001244
Vectorized RK	1148.933	0.000623

Backup: Contrived Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	251666.714	0.238081
Bird-KMP	755930.571	0.337132
Greedy BM	3019314.714	1.304504
Greedy KMP	1511864.143	0.751434
Naive	3019314.714	1.334626
Rabin-Karp	755387.000	0.708536
Vectorized RK	754836.429	0.353938

Backup: Pipeline Pseudocode

Experiment loop skeleton

1. Generate `text`, `pattern` for a scenario.
2. Run all algorithms and collect `(comparisons, match, wall_sec)`.
3. Append row records with scenario metadata.
4. Persist section CSV, then aggregate means by algorithm.
5. Plot comparisons and wall-clock for cross-method interpretation.

Backup: Reproducibility and Build

Regenerate metrics and compile slides

- `make metrics`
- `make figures`
- `make slides`

Key project files

- `scripts/recompute_metrics.py`
- `scripts/prepare_figures.py`
- `slides/main.tex` and `slides/sections/*.tex`
- `slides/generated/metrics_summary.csv`